

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

ARUNASHREE N (1BM24CS056)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by **ARUNASHREE N (1BM24CS056)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Syed Akram
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	4
2	Implement Johnson Trotter algorithm to generate permutations.	9
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	12
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	15
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	18
6	Implement 0/1 Knapsack problem using dynamic programming	21
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	25
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	29 32
9	Implement Fractional Knapsack using Greedy technique.	35
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	37
11	Implement "N-Queens Problem" using Backtracking.	40

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

LAB PROGRAM 1

Write program to obtain the Topological ordering of vertices in a given digraph.

```
#include<stdio.h>
#include<time.h>

void topologicalSort(int v,int adj[v][v]){
    int indegree[v], queue[v];
    int front=0,rear=-1;
    for(int i=0;i<v;i++){
        indegree[i]=0;
    }
    for(int i=0;i<v;i++){
        for(int j=0;j<v;j++){
            if(adj[i][j]==1){
                indegree[j]++;
            }
        }
    }
    for(int i=0;i<v;i++){
        if(indegree[i]==0){
            queue[++rear]=i;
        }
    }
    printf("\nTopological order:\t");
    while(front<=rear){
        int node=queue[front++];
        printf("%d ",node+1);
        for(int i=0;i<v;i++){
            if(adj[node][i]==1){
                indegree[i]--;
                if(indegree[i]==0){
                    queue[++rear]=i;
                }
            }
        }
    }
}

int main(){
    int v;
    printf("\nEnter the number of vertices: ");
    scanf("%d",&v);
    int adj[v][v];
    printf("\nEnter the adjacency matrix:\n");
    for(int i=0;i<v;i++){
        for(int j=0;j<v;j++){
            scanf("%d",&adj[i][j]);
        }
    }
    clock_t start, end;
```

```

start = clock();
topologicalSort(v,adj);
end = clock();
double time = (double)(end-start)/CLOCKS_PER_SEC;

printf("\nExecution time: %f\n",time);
return 0;
}

```

OUTPUT :

```
Enter the number of vertices: 4
```

```
Enter the adjacency matrix:
```

```
0 1 0 0
0 0 1 1
0 0 0 0
0 0 0 0
```

```
Topological order:      1 2 3 4
```

```
Execution time: 0.000000
```

```
Process returned 0 (0x0)   execution time : 11.624 s
```

```
Press any key to continue.
```

```
Enter the number of vertices: 4
```

```
Enter the adjacency matrix:
```

```
1 1 1 1
0 0 0 0
0 0 0 0
1 1 0 0
```

```
Topological order:
```

```
Execution time: 0.000000
```

```
Process returned 0 (0x0)   execution time : 12.654 s
```

```
Press any key to continue.
```

LEETCODE 207: COURSE SCHEDULE

```
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int* prerequisitesColSize) {  
    int inDegree[2000] = {0};  
    int queue[2000], front, rear, count;  
    front = rear = count = 0;  
  
    for (int i = 0; i < prerequisitesSize; i++) {  
        int course = prerequisites[i][0];  
        inDegree[course]++;  
    }  
  
    for (int i = 0; i < numCourses; i++) {  
        if (inDegree[i] == 0) {  
            queue[rear++] = i;  
        }  
    }  
  
    while (front < rear) {  
        int currCourse = queue[front++];  
        count++;  
        for (int i = 0; i < prerequisitesSize; i++) {  
            if (prerequisites[i][1] == currCourse) {  
                int depCourse = prerequisites[i][0];  
                inDegree[depCourse]--;  
  
                if (inDegree[depCourse] == 0) {  
                    queue[rear++] = depCourse;  
                }  
            }  
        }  
    }  
}
```

```
        }  
    }  
}  
  
return count == numCourses;  
}
```

OUTPUT:

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

numCourses =
2

prerequisites =
[[1,0]]

Output

true

Expected

true

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

numCourses =
2

prerequisites =
[[1,0],[0,1]]

Output

false

Expected

false

LAB PROGRAM 2

Implement Johnson Trotter algorithm to generate permutations.

```
#include <stdio.h>
#include <time.h>

int LR = 1;
int RL = 0;

void swap(int *a, int *b)
{
    int temp = *a;
    *a=*b;
    *b=temp;
}

int fact(int n)
{
    int res = 1;
    for (int i = 1; i <= n; i++)
        res = res * i;
    return res;
}

int Permutation(int a[], int dir[], int n)
{
    int prev = 0, mobile = 0, pos;
    for (int i = 0; i < n; i++)
    {
        if (dir[a[i] - 1] == RL && i != 0) {
            if (a[i] > a[i - 1] && a[i] > prev) {
                mobile = a[i];
                prev = mobile;
            }
        }

        if (dir[a[i] - 1] == LR && i != n - 1) {
            if (a[i] > a[i + 1] && a[i] > prev) {
                mobile = a[i];
                prev = mobile;
            }
        }
    }

    {
        for (int i = 0; i < n; i++)
            if (a[i] == mobile)
```

```

        pos = i + 1;
    }

    if (dir[a[pos - 1] - 1] == RL)
        swap(&a[pos - 1], &a[pos - 2]);

    else if (dir[a[pos - 1] - 1] == LR)
        swap(&a[pos], &a[pos - 1]);

    for (int i = 0; i < n; i++) {
        if (a[i] > mobile) {
            if (dir[a[i] - 1] == LR)
                dir[a[i] - 1] = RL;
            else if (dir[a[i] - 1] == RL)
                dir[a[i] - 1] = LR;
        }
    }

    for (int i = 0; i < n; i++)
        printf("%d", a[i]);
    printf("\n");
}

int main()
{
    int n, a[10], dir[10];
    printf("Enter the number of n: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        a[i] = i + 1;
        printf("%d", a[i]);
    }
    printf("\n");

    for (int i = 0; i < n; i++)
        dir[i] = RL;

    clock_t start, end;

    start = clock();

    for (int i = 1; i < fact(n); i++)
        Permutation(a, dir, n);

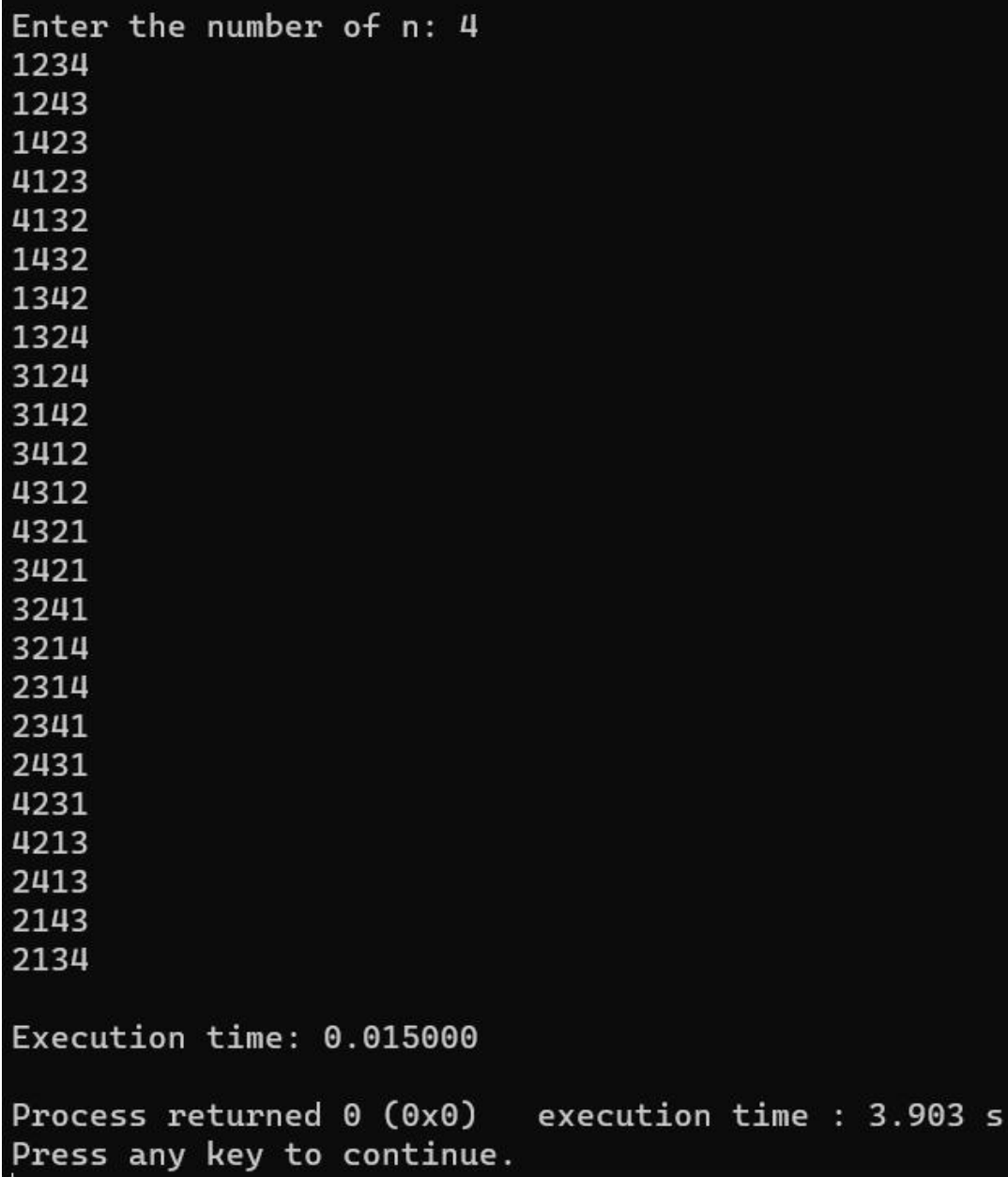
```

```
end = clock();

double time = (double) (end-start)/CLOCKS_PER_SEC;
printf("\nExecution time: %f\n", time);
return 0;

}
```

OUTPUT:



```
Enter the number of n: 4
1234
1243
1423
4123
4132
1432
1342
1324
3124
3142
3412
4312
4321
3421
3241
3214
2314
2341
2431
4231
4213
2413
2143
2134

Execution time: 0.015000

Process returned 0 (0x0)   execution time : 3.903 s
Press any key to continue.
```

LAB PROGRAM 3

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

```
#include<stdio.h>
```

```
#include<time.h>
```

```
void merge(int a[],int low, int mid, int high)
```

```
{  
    int i,j,k,b[50];  
    i=low;  
    j=mid+1;  
    k=low;
```

```
    while(i<=mid && j<=high)
```

```
    {  
        if(a[i]<a[j])  
        {  
            b[k]=a[i];  
            i++;  
            k++;  
        }  
        else  
        {  
            b[k]=a[j];  
            j++;  
            k++;  
        }  
    }
```

```
    while(i<=mid)
```

```
    {  
        b[k]=a[i];  
        i++;  
        k++;  
    }
```

```
    while(j<=high)
```

```
    {  
        b[k]=a[j];  
        j++;  
        k++;  
    }
```

```
    for(i=low;i<=high;i++)
```

```
    {  
        a[i]=b[i];  
    }  
}
```

```

void mergesort(int a[], int low, int high)
{
    int mid;

    if(low < high)
    {
        mid = (low + high)/2;

        mergesort(a, low, mid);
        mergesort(a, mid+1, high);

        merge(a, low, mid, high);
    }
}

int main()
{
    int a[50], n, i;

    printf("Enter number of elements: ");
    scanf("%d",&n);

    printf("Enter the elements:\n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }

    clock_t start, end;
    start=clock();
    mergesort(a,0,n-1);
    end=clock();

    double time =(double)(end-start)/CLOCKS_PER_SEC;
    printf("Sorted elements in ascending order:\n");
    for(i=0;i<n;i++)
    {
        printf("%d ",a[i]);
    }
    printf("\nExecution time: %f\n", time);
    return 0;
}

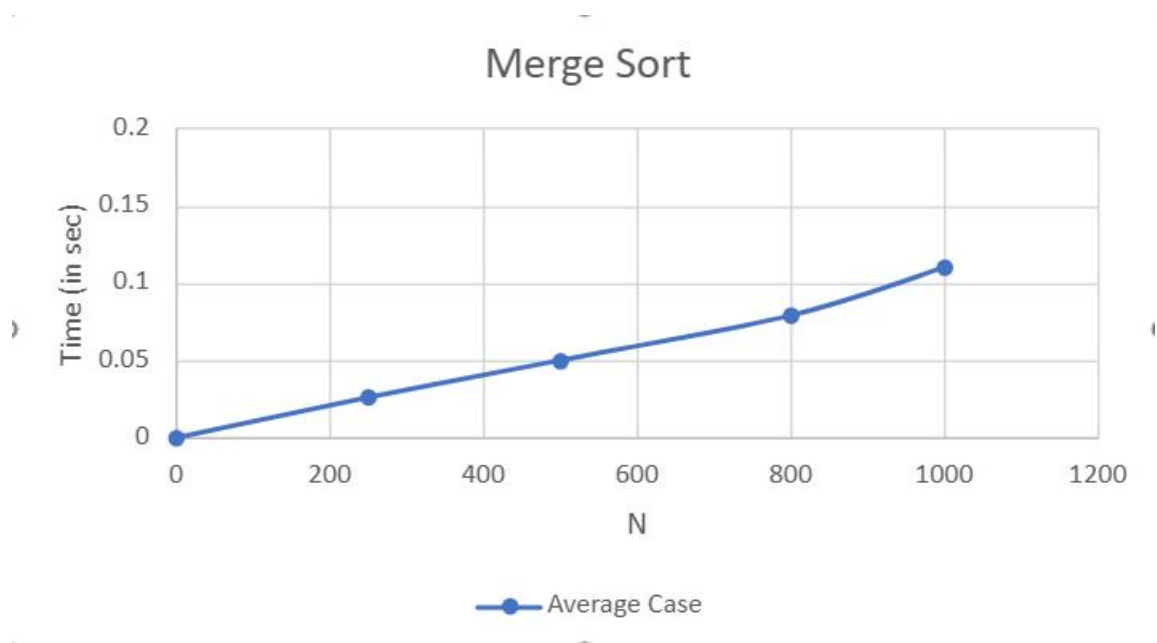
```

OUTPUT:

```
Enter number of elements: 10
Enter the elements:
36 15 47 255 98 1 32 61 24 3
Sorted elements in ascending order:
1 3 15 24 32 36 47 61 98 255
Execution time: 0.000000

Process returned 0 (0x0)   execution time : 21.667 s
Press any key to continue.
|
```

TIME COMPLEXITY:



LAB PROGRAM 4

Sort a given set of N integer elements using Quick Sort technique and compute its time taken

```
#include<stdio.h>
#include<time.h>
```

```
int partition(int a[], int low, int high)
{
    int i,j, key;
    i=low+1;
    j=high;
    key=a[low];
    while(1)
    {
        while(a[i]<=key&& i<high)
        {
            i++;
        }

        while(a[j]>key)
        {
            j--;
        }

        if(i<j)
        {
            int temp = a[i];
            a[i]=a[j];
            a[j]=temp;
        }
        else
        {
            int temp = a[low];
            a[low]=a[j];
            a[j]=temp;
            return j;
        }
    }
}
```

```
void quickSort(int a[], int low, int high)
{
    int j;
    if(low<high)
    {
        j=partition(a,low,high);
        quickSort(a,low,j-1);
        quickSort(a,j+1,high);
    }
}
```

```

    }

}

int main()
{
    int n, a[100];
    printf("Enter the number of elements in the array: ");
    scanf("%d",&n);
    printf("\nEnter the elements of the array: ");

    for(int i=0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }

    clock_t start, end;

    start=clock();
    quickSort(a,0,n-1);
    end=clock();
    printf("Sorted array:\n");
    for(int i=0;i<n;i++)
    {
        printf("%d\t", a[i]);
    }

    double time=(double)(end-start)/CLOCKS_PER_SEC;
    printf("\nExecution time: %f\n", time);
}

```


OUTPUT:

```
Enter the number of elements in the array: 10
Enter the elements of the array: 5 99 64 3 16 78 20 11 53 61
Sorted array:
3      5      11      16      20      53      61      64      78      99
Execution time: 0.000000

Process returned 0 (0x0)   execution time : 14.129 s
Press any key to continue.
```

TIME COMPLEXITY:



LAB PROGRAM 5

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

```
#include <stdio.h>
```

```
#include <time.h>
```

```
void heapify(int a[], int n, int p)
```

```
{  
    int c, item;  
    item = a[p];  
    c = 2 * p + 1;
```

```
    while (c < n)
```

```
    {  
        if (c + 1 < n)  
        {  
            if (a[c] < a[c + 1])  
            {  
                c++;  
            }  
        }  
    }
```

```
    if (item < a[c])
```

```
    {  
        a[p] = a[c];  
        p = c;  
        c = 2 * p + 1;
```

```
    }  
    else  
    {  
        break;
```

```
    }  
    a[p] = item;  
}
```

```
void build_heap(int a[], int n)
```

```
{  
    int i;
```

```
    for (i = (n - 1) / 2; i >= 0; i--)
```

```
    {  
        heapify(a, n, i);  
    }  
}
```

```
void heapsort(int a[], int n)
```

```

{
    int i, temp;

    build_heap(a, n);

    for (i = n - 1; i > 0; i--)
    {
        temp = a[0];
        a[0] = a[i];
        a[i] = temp;

        heapify(a, i, 0);
    }
    printf("Sorted array:\n");
    for (i = 0; i < n; i++)
    {
        printf("%d\t", a[i]);
    }
}

int main()
{
    clock_t start, end;
    int a[100], n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter array elements:\n");

    for (i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }

    start = clock();
    heapsort(a, n);
    end = clock();

    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;

    printf("\nExecution time: %f seconds\n", time_taken);

    return 0;
}

```

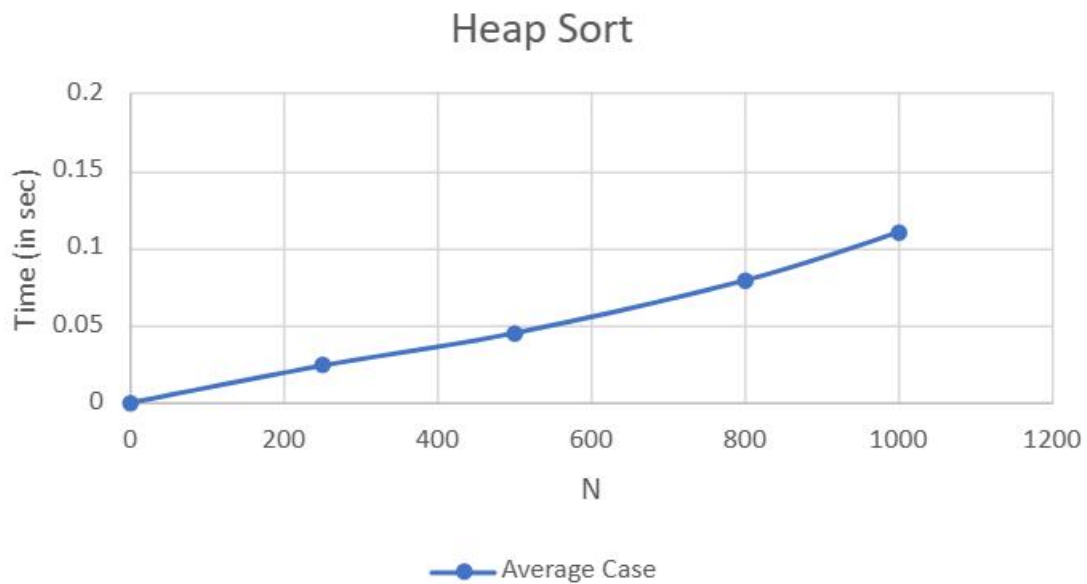
```
}
```

OUTPUT:

```
Enter number of elements: 10
Enter array elements:
35 6 49 99 2 54 62 28 520 65
Sorted array:
2      6      28      35      49      54      62      65      99      520
Execution time: 0.000000 seconds

Process returned 0 (0x0)   execution time : 23.361 s
Press any key to continue.
```

TIME COMPLEXITY:



LAB PROGRAM 6

Implement 0/1 Knapsack problem using dynamic programming.

```
#include<time.h>
#include<stdio.h>

int max(int a, int b)
{
    if (a>b)
        return a;
    else
        return b;
}

int main()
{
    int P[100], W[100], M, n, Table[100][100];
    printf("Enter the number of components: ");
    scanf("%d", &n);
    printf("Enter the maximum capacity of the knapsack: ");
    scanf("%d", &M);

    for(int i=1;i<=n;i++)
    {
        printf("Enter the weight and profit: ");
        scanf("%d%d", &W[i], &P[i]);
    }

    clock_t start, end;
    start = clock();
    for(int i=0;i<=n;i++)
    {
        for(int j=0;j<=M;j++)
        {
            if(i==0 || j==0)
            {
                Table[i][j]=0;
            }
            else if (j<W[i])
            {
                Table[i][j]=Table[i-1][j];
            }
            else
            {
                Table[i][j]=max(Table[i-1][j], P[i]+Table[i-1][j-W[i]]);
            }
        }
    }

    end = clock();
    printf("Maximum knap is %d", Table[n][M]);
    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;
```

```
printf("\nExecution time: %f seconds\n", time_taken);  
}
```

OUTPUT:

```
Enter the number of of components: 3  
Enter the maximum capacity of the knapsack: 4  
Enter the weight and profit: 2 16  
Enter the weight and profit: 2 6  
Enter the weight and profit: 1 18  
Maximum knap is 34  
Execution time: 0.000000 seconds  
  
Process returned 0 (0x0)    execution time : 9.533 s  
Press any key to continue.
```

LEETCODE 801: Minimum Swaps To Make Sequences Increasing

```
int minSwap(int* nums1, int nums1Size, int* nums2, int nums2Size)
{
    int a = 0, b = 1;
    for (int i = 1; i < nums1Size; ++i) {
        int x = a, y = b;
        if (nums1[i - 1] >= nums1[i] || nums2[i - 1] >= nums2[i]) {
            a = y;
            b = x + 1;
        } else {
            b = y + 1;
            if (nums1[i - 1] < nums2[i] && nums2[i - 1] < nums1[i]) {
                a = fmin(a, y);
                b = fmin(b, x + 1);
            }
        }
    }
    return fmin(a, b);
}
```

OUTPUT:

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

nums1 =
[1,3,5,4]

nums2 =
[1,2,3,7]

Output

1

Expected

1

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

nums1 =
[0,3,5,8,9]

nums2 =
[2,1,4,6,9]

Output

1

Expected

1

LAB PROGRAM 7

Implement All Pair Shortest paths problem using Floyd's algorithm.

```
#include <stdio.h>
#include<time.h>
#define INF 99

void floyd(int A[][10], int n)
{
    int i,j,k;
    for(k = 0; k < n; k++) {
        for(i = 0; i < n; i++) {
            for(j = 0; j < n; j++) {
                if(A[i][k] + A[k][j] < A[i][j]) {
                    A[i][j] = A[i][k] + A[k][j];
                }
            }
        }
    }
    printf("\nShortest Distance Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            if(A[i][j] == INF)
                printf("INF ");
            else
                printf("%d ", A[i][j]);
        }
        printf("\n");
    }
}

int main() {
    int n, i, j, k;
    int cost[10][10], A[10][10];
    printf("Enter the number of vertices: ");
    scanf("%d", &n);
    printf("Enter the cost matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
            A[i][j] = cost[i][j];
        }
    }
    clock_t start, end;

    start=clock();
    floyd(A, n);
    end=clock();

    double time=(double)(end-start)/CLOCKS_PER_SEC;
    printf("Execution time: %f", time);
    return 0;
}
```

}

OUTPUT:

```
Enter the number of vertices: 5
Enter the cost matrix:
0 4 99 5 99
99 0 1 99 6
2 99 0 3 99
99 99 1 0 2
1 99 99 4 0

Shortest Distance Matrix:
0 4 5 5 7
3 0 1 4 6
2 6 0 3 5
3 7 1 0 2
1 5 5 4 0
Execution time: 0.016000
Process returned 0 (0x0)    execution time : 29.066 s
Press any key to continue.
```

LEETCODE 287: Find the Duplicate Number

```
int findDuplicate(int* nums, int numsSize)
{
    int slow1, fast, slow2;
    slow1=fast=slow2=0;
    while(true)
    {
        slow1=nums[slow1];
        fast=nums[nums[fast]];
        if(slow1==fast)
        {
            break;
        }
    }
    while(true)
    {
        slow1=nums[slow1];
        slow2=nums[slow2];
        if(slow1==slow2)
        {
            return slow2;
        }
    }
}
```

OUTPUT:

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2 ☒ Case 3

Input

```
nums =  
[1,3,4,2,2]
```

Output

```
2
```

Expected

```
2
```

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2 ☒ Case 3

Input

```
nums =  
[3,1,3,4,2]
```

Output

```
3
```

Expected

```
3
```

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2 ☒ Case 3

Input

```
nums =  
[3,3,3,3,3]
```

Output

```
3
```

Expected

```
3
```

LAB PROGRAM 8

a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

```
#include<stdio.h>
#include<time.h>

int minKey(int key[], int set[], int n)
{
    int min=9999, min_idx;
    for(int i=0;i<n;i++)
    {
        if(set[i]==0 && key[i]<min)
        {
            min=key[i];
            min_idx=i;
        }
    }
    return min_idx;
}

int prim(int graph[][100], int n)
{
    int key[100], set[100], parent[100] ;

    for(int i=0;i<n;i++)
    {
        key[i]=999;
        set[i]=0;
    }

    key[0]=0;
    parent[0]=-1;

    for(int k=0;k<n-1;k++)
    {
        int u = minKey(key,set,n);
        set[u]=1;

        for(int i=0;i<n;i++)
        {
            if((graph[u][i])&&set[i]==0 && graph[u][i]<key[i])
            {
                parent[i]=u;
                key[i]=graph[u][i];
            }
        }
    }
    int mincost=0;
    printf("EDGE\tWEIGHT\n");
```

```

for(int i=1;i<n;i++)
{
    printf("%d-%d\t%d\n", parent[i],i,graph[parent[i]][i]);
    mincost+=graph[parent[i]][i];
}
printf("The minimum cost is %d", mincost);
return 0;
}
int main()
{
    int graph[100][100],n;

    printf("Enter the number of vertices in the graph: ");
    scanf("%d", &n);

    printf("\nEnter the adjacency matrix:\n");

    for(int i=0;i<n;i++)
    {
        for(int j=0;j<n;j++)
        {
            scanf("%d", &graph[i][j]);
        }
    }

    clock_t start, end;

    start=clock();
    prim(graph, n);
    end=clock();

    double time=(double)(end-start)/CLOCKS_PER_SEC;
    printf("\nExecution time: %f\n", time);
}

```

OUTPUT:

```
Enter the number of vertices in the graph: 5

Enter the adjacency matrix:
0 2 0 6 0
2 0 3 8 5
0 3 0 0 7
6 8 0 0 9
0 5 7 9 0
EDGE      WEIGHT
0-1        2
1-2        3
0-3        6
1-4        5
The minimum cost is 16
Execution time: 0.000000

Process returned 0 (0x0)    execution time : 18.377 s
Press any key to continue.
```

LAB PROGRAM 8

b)Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAX 30

typedef struct {
    int u, v, w;
} Edge;

Edge edges[MAX], result[MAX];
int parent[MAX];

int find(int i) {
    while (parent[i] != i)
        i = parent[i];
    return i;
}

void setUnion(int i, int j) {
    parent[i] = j;
}

void sortEdges(int e) {
    int i, j;
    Edge temp;
    for (i = 0; i < e - 1; i++) {
        for (j = 0; j < e - i - 1; j++) {
            if (edges[j].w > edges[j + 1].w) {
                temp = edges[j];
                edges[j] = edges[j + 1];
                edges[j + 1] = temp;
            }
        }
    }
}

void kruskal(Edge edges[], int n, int m)
{
    int i,u,v, count=0, cost=0;
    for (i = 0; i < n; i++)
        parent[i] = i;
```



```

sortEdges(m);

for (i = 0; i < m && count < n - 1; i++) {
    u = find(edges[i].u);
    v = find(edges[i].v);

    if (u != v) {
        result[count++] = edges[i];
        cost += edges[i].w;
        setUnion(u, v);
    }
}

printf("\nMinimum Spanning Tree:\n");
for (i = 0; i < count; i++) {
    printf("%d-%d\t%d\n", result[i].u, result[i].v, result[i].w);
}

printf("Total cost = %d\n", cost);

}
int main() {
    int n, m, i, j, u, v;
    int count = 0, cost = 0;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter number of edges: ");
    scanf("%d", &m);

    printf("Enter edges (u v weight):\n");
    for (i = 0; i < m; i++) {
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);
    }

    clock_t start, end;
    start=clock();
    kruskal(edges, n,m);
    end=clock();
    double time = (double)(end-start)/CLOCKS_PER_SEC;
    printf("\nExecution time: %f\n", time);
    return 0;
}

```

OUTPUT

```
Enter number of vertices: 5
Enter number of edges: 5
Enter edges (u v weight):
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4

Minimum Spanning Tree:
2-3      4
0-3      5
0-1     10
Total cost = 19

Execution time: 0.000000

Process returned 0 (0x0)    execution time : 22.803 s
Press any key to continue.
|
```

LAB PROGRAM 9

Implement fractional knapsack problem using Greedy technique.

```
#include<time.h>
struct Item {
    int value, weight;
    float ratio;
};
void swap(struct Item a[], int i, int j) {
    struct Item temp = a[i];
    a[i] = a[j];
    a[j] = temp;
}
void sort(struct Item a[], int n) {
    for(int i = 0; i < n - 1; i++) {
        for(int j = 0; j < n - i - 1; j++) {
            if(a[j].ratio < a[j + 1].ratio) {
                swap(a, j, j + 1);
            }
        }
    }
}
float fractionalKnapsack(struct Item a[], int n, int capacity) {
    float total = 0.0;

    for(int i = 0; i < n; i++) {
        if(capacity >= a[i].weight) {
            total += a[i].value;
            capacity -= a[i].weight;
        } else {
            total += a[i].ratio * capacity;
            break;
        }
    }

    return total;
}
int main() {
    int n, capacity;

    printf("Enter number of items: ");
    scanf("%d", &n);

    struct Item a[n];
    for(int i = 0; i < n; i++) {
        printf("Enter value and weight of item %d: ", i + 1);
        scanf("%d %d", &a[i].value, &a[i].weight);
        a[i].ratio = (float)a[i].value / a[i].weight;
    }
}
```

```

printf("Enter knapsack capacity: ");
scanf("%d", &capacity);
sort(a, n);
float maxVal = fractionalKnapsack(a, n, capacity);
printf("Maximum value = %.2f\n", maxVal);

clock_t start, end;
start=clock();
fractionalKnapsack(a,n,capacity);
end=clock();
double time=(double)(end-start)/CLOCKS_PER_SEC;
printf("Execution time:%f",time);
return 0;

}

```

OUTPUT:

```

Enter number of items: 3
Enter value and weight of item 1: 25 18
Enter value and weight of item 2: 24 15
Enter value and weight of item 3: 15 10
Enter knapsack capacity: 20
Maximum value = 31.50
Execution time:0.000000
Process returned 0 (0x0)    execution time : 36.529 s
Press any key to continue.

```

LAB PROGRAM 10

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

```
#include <stdio.h>
#include <time.h>
```

```
int minDist(int dist[], int vis[], int n) {
    int min = 10000, min_idx;
```

```
    for (int i = 0; i < n; i++) {
        if (vis[i] == 0 && dist[i] <= min) {
            min = dist[i];
            min_idx = i;
        }
    }
    return min_idx;
}
```

```
void dijkstra(int graph[][100], int src, int n) {
    int dist[100];
    int vis[100];
```

```
    for (int i = 0; i < n; i++) {
        dist[i] = 10000;
        vis[i] = 0;
    }
```

```
    dist[src] = 0;
```

```
    for (int count = 0; count < n - 1; count++) {
        int u = minDist(dist, vis, n);
        vis[u] = 1;
```

```
        for (int v = 0; v < n; v++)
        {
            if (!vis[v] && graph[u][v] && dist[u] != 10000 && dist[u] + graph[u][v] < dist[v])
            {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }
```

```
    printf("Vertex \t Distance from Source\n");
```

```

        for (int i = 0; i < n; i++) {
            printf("%d \t\t %d\n", i, dist[i]);
        }
    }

int main() {
    int graph[100][100],n;
    printf("Enter the number of vertices in the graph: ");
    scanf("%d",&n);
    printf("Enter weight matrix: \n");
    for(int i=0;i<n;i++)
    {
        for(int j=0;j<n;j++)
        {
            scanf("%d", &graph[i][j]);
        }
    }

    int source = 0;
    clock_t start, end;
    start=clock();
    dijkstra(graph, source, n);
    end=clock();

    double time=(double)(end-start)/CLOCKS_PER_SEC;
    printf("\nExecution time: %f\n", time);

    return 0;
}

```

OUTPUT:

```
Enter the number of vertices in the graph: 5
Enter weight matrix:
0 10 0 30 100
10 0 50 0 0
0 50 0 20 10
30 0 20 0 60
100 0 10 60 0
Vertex    Distance from Source
0          0
1          10
2          50
3          30
4          60

Execution time: 0.000000

Process returned 0 (0x0)    execution time : 78.941 s
Press any key to continue.
```

LAB PROGRAM 11

Implement “N-Queens Problem” using Backtracking.

```
#include<stdio.h>
#include<time.h>

int isSafe(int board[][10], int row, int col, int n)
{
    int i,j;
    for(i=0;i<col;i++)
    {
        if(board[row][i])
            return 0;
    }
    for(i=row,j=col;i>=0 && j>=0;i--,j--)
    {
        if(board[i][j])
            return 0;
    }
    for(i=row,j=col;i<n && j>=0;i++,j--)
    {
        if(board[i][j])
            return 0;
    }
}

int printBoard(int board[10][10], int n)
{
    printf("\nBoard: \n");
    for(int i=0;i<n; i++)
    {
        for(int j=0;j<n; j++)
        {
            printf(" %d ", board[i][j]);
        }
        printf("\n");
    }
}

int place(int board[][10], int col, int n)
{
    if(col>=n)
    {
        return 1;
    }

    for(int i=0;i<n;i++)
    {
        if (isSafe(board, i, col, n))
        {
            board[i][col]=1;
            if(place(board,col+1, n ))
                return 1;
        }
    }
}
```



```

        board[i][col]=0;
    }
}

return 0;
}

int main()
{
    int n;
    int board[10][10]={0};
    printf("Enter the number of queens to place: ");
    scanf("%d", &n);

    if(!place(board, 0,n))
    {
        printf("\nNo solution found\n");
    }
    clock_t start, end;
    start=clock();
    printBoard(board,n);
    end=clock();

    double time = (double)(end-start)/CLOCKS_PER_SEC;
    printf("\nExecution time: %f\n", time);
    return 0;
}

```

OUTPUT:

```
Enter the number of queens to place: 10
```

```
Board:
```

```
1 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 1 0 0
0 1 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 1 0
0 0 0 0 0 1 0 0 0 0
0 0 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 1
0 0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 1 0 0 0
0 0 0 0 1 0 0 0 0 0
```

```
Execution time: 0.048000
```

```
Process returned 0 (0x0)   execution time : 2.409 s
Press any key to continue.
```

```
Enter the number of queens to place: 3
```

```
No solution found
```

```
Board:
```

```
0 0 0
0 0 0
0 0 0
```

```
Execution time: 0.000000
```

```
Process returned 0 (0x0)   execution time : 2.883 s
Press any key to continue.
```